

Phase 1: Core Fundamentals & The Type System

Understanding how data is stored and evaluated is the foundation of C#.

1. What is the difference between Value Types and Reference Types?
2. Explain Boxing and Unboxing. Why are they expensive?
3. What is the difference between var, dynamic, and object in C#?
4. What is the difference between ==, .Equals(), and ReferenceEquals()?
5. Explain string vs StringBuilder. When should you use each?
6. Explain Nullable Value Types and Nullable Operators (?, ??, ??=, ?.).
7. What are Nullable Reference Types (NRT) introduced in C# 8?

Phase 2: Parameters, Methods & Flow

How data is passed and how logic flows through a modern C# application.

8. What is the difference between ref and out parameters? Also explain in.
9. Explain Extension Methods and their internal working.
10. Explain advanced Pattern Matching in modern C#.

Phase 3: Object-Oriented Programming (OOP) Foundations

Structuring state and behavior safely.

11. Explain the Four Pillars of OOP with real examples.
12. What is abstraction?
13. What is encapsulation?
14. What is inheritance? Why is it important?
15. What are the different types of inheritance?
16. How to prevent a class from being inherited?
17. What is polymorphism and what are its types?
18. What is method overloading? In how many ways can a method be overloaded?
19. What is the difference between overloading and overriding?
20. What is the difference between method overriding and method hiding?
21. Abstract Class vs Interface - when to use each?
22. Why to even create interfaces?
23. Can an interface have a constructor?
24. Can you create an instance of an abstract class or an interface?
25. Explain sealed, virtual, override, and new modifiers.
26. Compare class, struct, and record in C# 9+.
27. Explain Records vs Classes for Domain Modelling.
28. What is the difference between Static, Instance, and Singleton?

29. What are the advantages and limitations of OOPs?

Phase 4: Functional Concepts & Generics

Abstracting logic and types for reusability.

- 17. What are Delegates, Func, Action, and Predicate?
- 18. Explain Events vs Delegates.
- 19. Explain Generics and Constraints in depth.
- 20. Explain Covariance and Contravariance in C# generics.

Phase 5: Exception Handling

Managing predictable and unpredictable failures.

- 21. Explain best practices for Exception Handling in C#.
- 22. What is the difference between throw and throw ex?
- 23. Explain finally, try/catch/finally semantics, and when finally doesn't run.
- 24. Explain Exception Filters (when clause).
- 25. What are Custom Exceptions and when to create them?

Phase 6: Memory Lifecycle & Resource Management

Understanding what happens under the hood with memory allocation and cleanup.

- 26. Explain Stack vs Heap in .NET.
- 27. Explain Garbage Collection generations in .NET.
- 28. Explain GC modes: Workstation vs Server, Concurrent vs Background.
- 29. What is Short Object Heap (SOH) & Large Object Heap (LOH) and why does it matter?
- 30. What causes Memory Leaks in managed .NET code?
- 31. What are Weak References and when to use them?
- 32. Explain IDisposable and the using statement/declaration.
- 33. Explain IDisposable vs Finalizer. When do you need a finalizer?
- 34. Explain the full Dispose Pattern (with finalizer).

Phase 7: Asynchronous Programming

Scaling applications through non-blocking operations.

- 35. What is the difference between Task, Task, ValueTask, and Thread?
- 36. Explain async/await internals (state machine).
- 37. Explain Task.Run vs Task.Factory.StartNew vs async Task.
- 38. Explain Task.WhenAll vs Task.WaitAll.
- 39. What is ConfigureAwait(false) and when should you use it?
- 40. What is a Cancellation Token and how is it used?
- 41. Explain IAsyncEnumerable and await foreach.

Phase 8: Concurrency, Synchronization & Pipelines

Protecting shared state and building multi-threaded data pipelines.

- 42. What are the differences between lock, Monitor, SemaphoreSlim, and Mutex?
- 43. Explain lock internals and why lock object choice matters.
- 44. What is Channel and when to use it?
- 45. Explain the Producer/Consumer pattern with Channel.

Phase 9: High-Performance C#

Minimizing allocations and accessing raw memory for latency-critical code.

- 46. What are Span and Memory, and when should you use them?
- 47. Explain ArrayPool and MemoryPool.
- 48. What are readonly struct and ref struct? Why do they matter?
- 49. Explain Unsafe Code and stackalloc.

Phase 10: Architectural Design & SOLID Principles

Lead-level concepts for designing maintainable systems.

- 50. Explain SOLID Single Responsibility Principle (SRP).
- 51. Explain Open/Closed Principle (OCP).
- 52. Explain Liskov Substitution Principle (LSP).
- 53. Explain Interface Segregation Principle (ISP).
- 54. Explain Dependency Inversion Principle (DIP).
- 55. What is the difference between Composition and Inheritance? Why prefer composition?
- 56. Explain Immutability Strategies in C#.

Phase 11: Metaprogramming & Modern Language Evolution

Manipulating the compiler and generating code at runtime or compile-time.

- 57. What is Reflection and when should it be avoided?
- 58. Explain Expression Trees.
- 59. What are Source Generators and how do they compare with Reflection?
- 60. What are Interceptors (C# 12)?

Topic-wise Categorization

Core OOPs Principles: Classes, Objects, Abstraction, Encapsulation, Inheritance, Polymorphism

Architecture & Contracts: Interfaces, Abstract Classes, Method Overriding, Method Hiding (new), Preventing Inheritance (sealed, static)

Memory & Performance: Boxing vs Unboxing, String vs StringBuilder, IQueryable vs IEnumerable

Type Safety & Extensibility: Generics, Extension Methods, var vs dynamic, is vs as

Control Flow & State: Loops (while, for, foreach), break/continue, Exception Handling (try, catch, finally, throw), readonly vs const

Data Structures: Arrays, ArrayList, Hashtable, Generic Collections, Concurrent Collections

Advanced Construct Operations: Constructors (Default, Parameterized, Static, Private, Copy), ref/out, params

Event-Driven Programming: Delegates, Multicast Delegates, Anonymous Delegates, Events

Beginner → Advanced Learning Flow

Beginner (Foundation): Focus on the definition of classes, objects, and the 4 pillars of OOPs

Master basic loops, strings, and default access modifiers (internal, public, private)

Intermediate (Implementation): Understand Overloading vs Overriding

Grasp the differences between Abstract Classes and Interfaces, and Collections (Array vs ArrayList vs Hashtable)

Dive into basic Exception Handling (try/catch/finally)

Advanced (Optimization & Types): Learn memory implications like Boxing/Unboxing and how Generics solve performance issues

Explore type handling (out/ref, var/dynamic) and iterative interfaces (IEnumerable vs IEnumerator)

Architect-Level (Design & Scalability): Master interface-driven enterprise design (Contracts)

Implement thread-safe Concurrent Collections

Utilize IQueryable for SQL optimization

Implement Singleton patterns via Private Constructors

Build secure, event-driven systems using Multicast Delegates and Events

Maintain unmanaged resources via IDisposable and the using block

Interlinked Question Chains

The Inheritance & Contract Chain: What is Inheritance?

→ How to prevent it? (sealed keyword)

→ How is an Abstract Class different from an Interface?

→ When to use which?

→ Why even create interfaces? (To force enterprise contracts)

The Polymorphism & Versioning Chain: What is Polymorphism?

→ Difference between Compile-time (Overloading) and Run-time (Overriding)?

→ Difference between Overriding and Method Hiding? (Using the new keyword to sever the relationship with the base class implementation)

The Error Management Chain: How to handle exceptions?

→ Can we have multiple catch blocks? (Yes, for different exception types)

→ Can we have a try without a catch? (Yes, if finally is present for mandatory teardown)

→ What is the difference between throw and throw ex?

The Iteration & Query Chain: What are Collections?

→ How does foreach work? (Via IEnumerable)

→ How is IEnumerable structured? (It exposes IEnumerator)

→ Why switch to IQueryable for databases? (Server-side vs. Client-side filtering)

Scenario-Based Questions

The Undefined Employee Scenario: You are hiring an employee whose project and manager are not yet known. Do you use an Abstract Class or an Interface? Response: Interface. Abstract classes are for defined/concrete behaviors (like their role), whereas Interfaces are for behaviors that must exist but are implemented entirely differently by derived classes

The Database Failure Scenario: A connection is opened, but an arithmetic exception occurs during processing. How do you prevent memory leaks? Response: Place the .Close() or .Dispose() method inside a finally block, OR wrap the connection logic inside a using statement (which automatically calls IDisposable)

The String Modification Scenario: You are inside a loop modifying a string 1,000 times. What data type should you use? Response: StringBuilder. Because standard String is immutable, modifying it 1,000 times creates 1,000 new objects in memory. StringBuilder manipulates the same memory reference, drastically improving performance

Tricky Questions

Can an interface have a constructor? Answer: No. Constructors initialize objects, and interfaces cannot be instantiated. Only derived classes implement the interface

Can we execute multiple catch blocks at once? Answer: No. Even if multiple catch blocks are written, only the first block matching the specific exception type will execute

Do we need to initialize an out parameter before passing it? Answer: No, the out parameter does not need initialization before passing, but the method must initialize it before returning. Conversely, ref parameters must be initialized before passing

Why does an Abstract class need a constructor if it can't be instantiated? Answer: When a derived child class creates an object, the base class constructor is automatically called first. The abstract class constructor is executed during the initialization of the derived class

Ambiguity-Based Questions

is vs as operator: Candidates often confuse these. is returns a Boolean (true/false) to check if an object is of a specific type. as actually attempts to convert/cast the object to the type, returning null if the cast fails

readonly vs const: Is a constant always evaluated at compile time? Yes, const is a compile-time constant and can only be set at declaration. readonly is a runtime constant and can be set at declaration or within the constructor

Method Overloading vs Overriding default state: Candidates forget that overloading happens in the same class (compile-time) while overriding requires inheritance (runtime) and the use of virtual/override keywords

Real-World Architecture Questions

Database Query Optimization: Why use IQueryable instead of IEnumerable when dealing with Entity Framework / SQL? Using IEnumerable pulls the entire database table across the network into application memory, then filters it. IQueryable translates the filter into a SQL query and executes it on the database server, vastly reducing network overhead and memory consumption

Enterprise Contracts (Compliance): How do you ensure all developer teams create classes that conform to company standards? By defining Interfaces (e.g., IEmployee with SetSalary and SetProject). If a derived class fails to implement the interface methods, the project will throw a compile-time error, enforcing architectural consistency

Resource Management: How do you guarantee the destruction of unmanaged resources? Implement the IDisposable interface on managed classes and utilize the using statement, which guarantees the Dispose() method is fired even if an exception breaks the application flow

Coding Questions

Implementing Generic Equality: Write a AreEqual<T>(T a, T b) method using generics to compare values. This eliminates the need for Object type casting, preventing the expensive memory overhead of Boxing/Unboxing

Extending Sealed Classes: Write an extension method to add a RightSubstring() feature to the String class. Solution: Create a static class containing a static method, and use the this keyword on the first parameter (e.g., this string value) to bind it

Multicast Delegate Configuration: How do you bind multiple methods to a single delegate caller? Solution: Instantiate the delegate with the first method (calc = new Calculator(Add)), then use the += operator to chain subsequent methods (calc += Multiply). Firing the delegate will execute both sequentially

Senior-Level Deep Dive Questions

The Hidden Mechanics of foreach: A foreach loop does not inherently know how to iterate a List. It relies on the List implementing IEnumerable, which exposes the IEnumerator interface. The IEnumerator contains the actual state machine logic (MoveNext, Current) to traverse the collection

Compile-Time vs Run-Time Resolution: In Polymorphism, method overloading is resolved by the compiler generating distinct IL (Intermediate Language) signatures based on parameter counts/types. Method overriding is resolved at runtime via virtual tables, dynamically looking up the derived class method

Boxing/Unboxing Memory Penalties: Boxing converts a value type (stored on the Stack) to a reference type (stored on the Heap), and Unboxing forces an explicit cast back. This involves memory allocation and garbage collection overhead, heavily impacting high-performance backend systems

Hidden Concepts Detected

Singleton Pattern Implication: A private constructor prevents instantiation and inheritance. The hidden implication here is its primary use case in Enterprise Architecture: The Singleton Design Pattern, ensuring only one instance of a configuration manager or database context exists

V-Tables & Method Hiding: The new keyword for method hiding deliberately suppresses virtual table lookups for polymorphism, breaking the inheritance chain for that specific method and forcing the compiler to use the reference type's implementation instead of the instantiated object's implementation

Stateful Event Wrappers: Events act as a secure wrapper around delegates. The hidden concept is access restriction; while a client can invoke a raw delegate directly or clear its invocation list, an event restricts clients to only subscribing (+=) or unsubscribing (-=), protecting the publisher's state

Common Interview Traps

The Stack Trace Wipe Trap: An interviewer asks, "Inside a catch block, should I use throw or throw ex?" **Trap Answer:** throw ex. **Correct Answer:** Never use throw ex. It resets the stack trace to the catch block's line number, permanently deleting the origin of the bug. Simply use throw to preserve the original stack trace

Multiple Inheritance Trap: An interviewer asks, "How do we achieve multiple inheritance with Abstract Classes?" **Correct Answer:** You cannot. C# only allows multiple inheritance through Interfaces

Default Modifier Trap: "What is the default access modifier of a class member vs the class itself?" **Answer:** The default access modifier of a class is internal. The default modifier of a class member (if unspecified) is private

Enterprise-Level Design Discussions

Concurrency & Multi-Threading: In large backend systems, standard collections (List, Dictionary) are not thread-safe. Architects must utilize Concurrent Collections (System.Collections.Concurrent) to manage multi-threaded data manipulation without locking deadlocks

Static Containers for Utilities: In enterprise applications, utility functions (e.g., Math operations, configuration parsers) should be grouped in static classes. This prevents unnecessary object instantiation in memory and locks down inheritance to prevent unauthorized behavioral overrides

Dynamic Late Binding: Integrating with external APIs, COM objects, or weakly-typed data sources (like dynamic JSON) requires the dynamic keyword. Unlike var (which is strictly typed at compile-time), dynamic bypasses compile-time checks and resolves types completely at runtime

Most Important Questions for Product Companies

What is the memory difference between String and StringBuilder? (Critical for processing high-throughput string manipulation without causing Garbage Collection pauses)

Explain IEnumerable vs IQueryable. (Critical for Backend/Database engineers to prevent pulling millions of rows into RAM)

Why and how do you use the using block? (Critical for demonstrating knowledge of unmanaged resource cleanup and memory leak prevention)

How do Generics prevent Boxing/Unboxing? (Critical for building high-performance, type-safe API endpoints and data structures)

Frequently Repeated Concepts

Compile-Time vs Run-Time Evaluation: This distinction appears constantly throughout C# OOPs: Overloading (Compile) vs Overriding (Run)

const (Compile) vs readonly (Run)

var (Compile) vs dynamic (Run)

Security and Abstraction Wrappers: Properties wrap Fields

Events wrap Delegates

and Interfaces wrap Application logic into Contracts

Memory Efficiency: Avoiding object recreation (StringBuilder)

avoiding heap allocation (Generics vs Boxing)

and immediate memory release (IDisposable / using)